Lab 4

Deploying a Multi-Tier Application with Docker Compose (Task Manager)

Item	Details
Course	Cloud Computing
Instructor	
Student name	Ahmed Salah
Student ID / Group	202200212
Lab date	28 October 2026
Environment	Docker Engine 29.8.2, Compose v5.6.0 (Docker Desktop, WSL2); commands run in Ubuntu 24.04

Note on the environment: the commands were run in an Ubuntu 24.04 (WSL) terminal connected to Docker Desktop, so 'docker' works without 'sudo'. The frontend is published on 8081 instead of 8080 because another program already uses port 8080 on this PC (the fix from the lab's troubleshooting table).

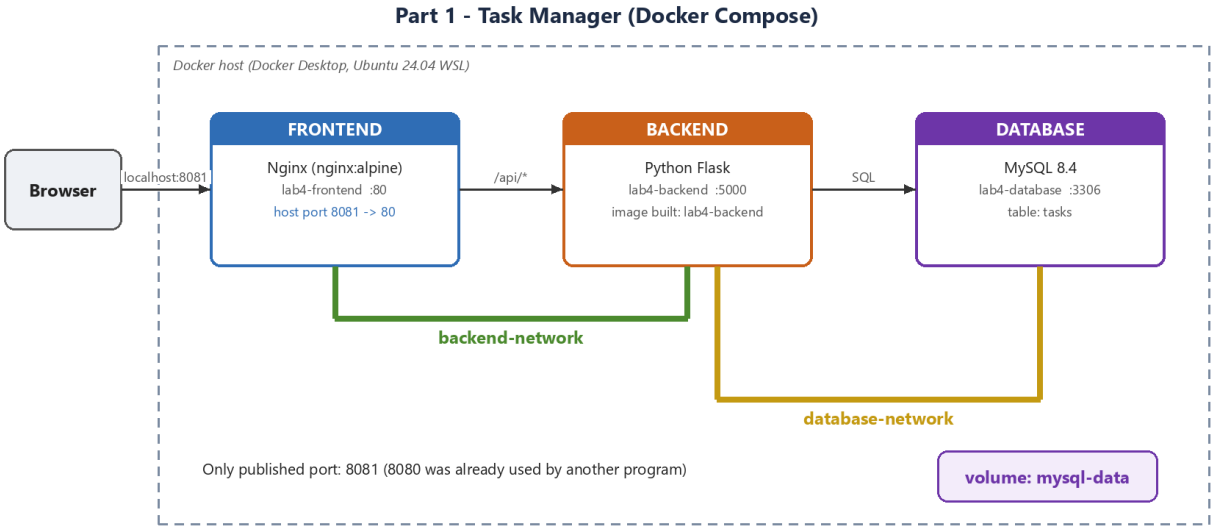


Figure 1: The deployed Task Manager: three containers, two networks, one volume

Screenshots

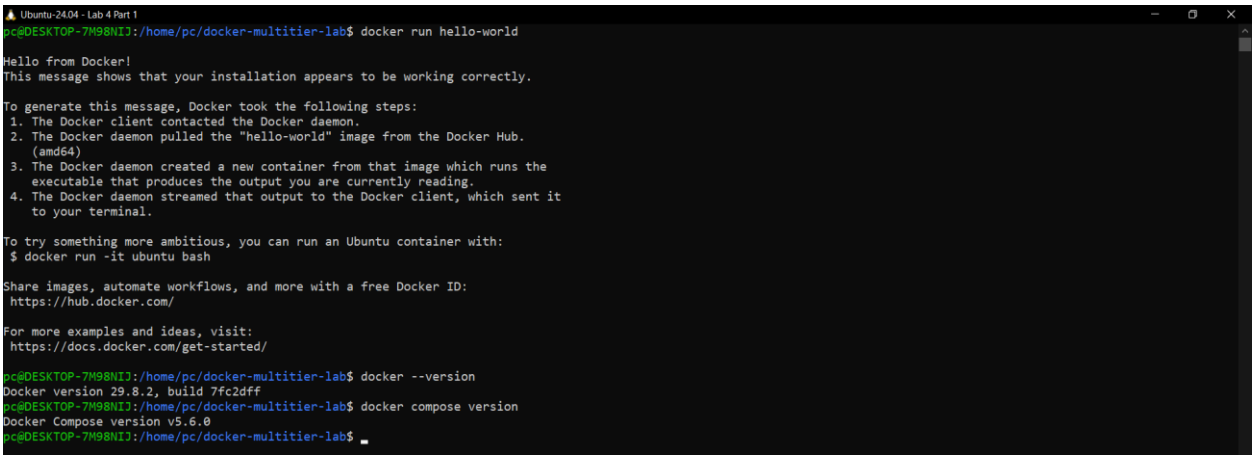


Figure 2: Screenshot 1 - hello-world success message, Docker and Compose versions

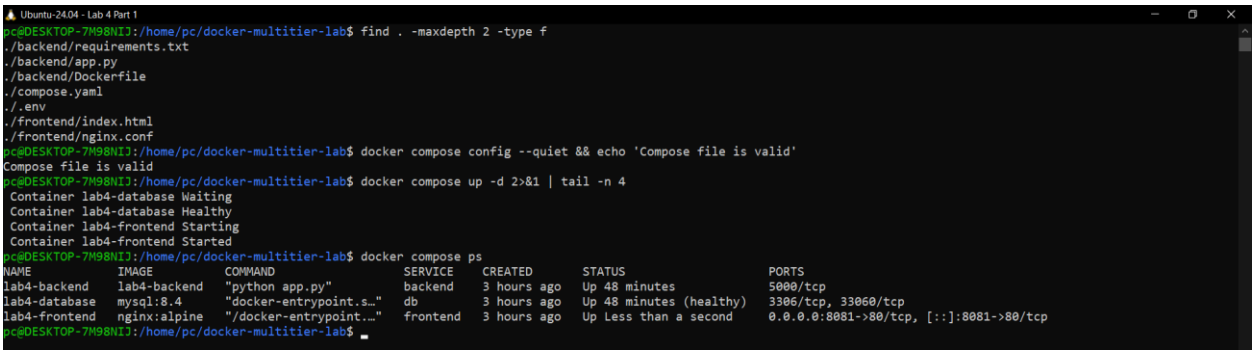


Figure 3: Screenshot 2 - the 7 project files, valid compose file, and docker compose ps: frontend and backend running, db running (healthy)

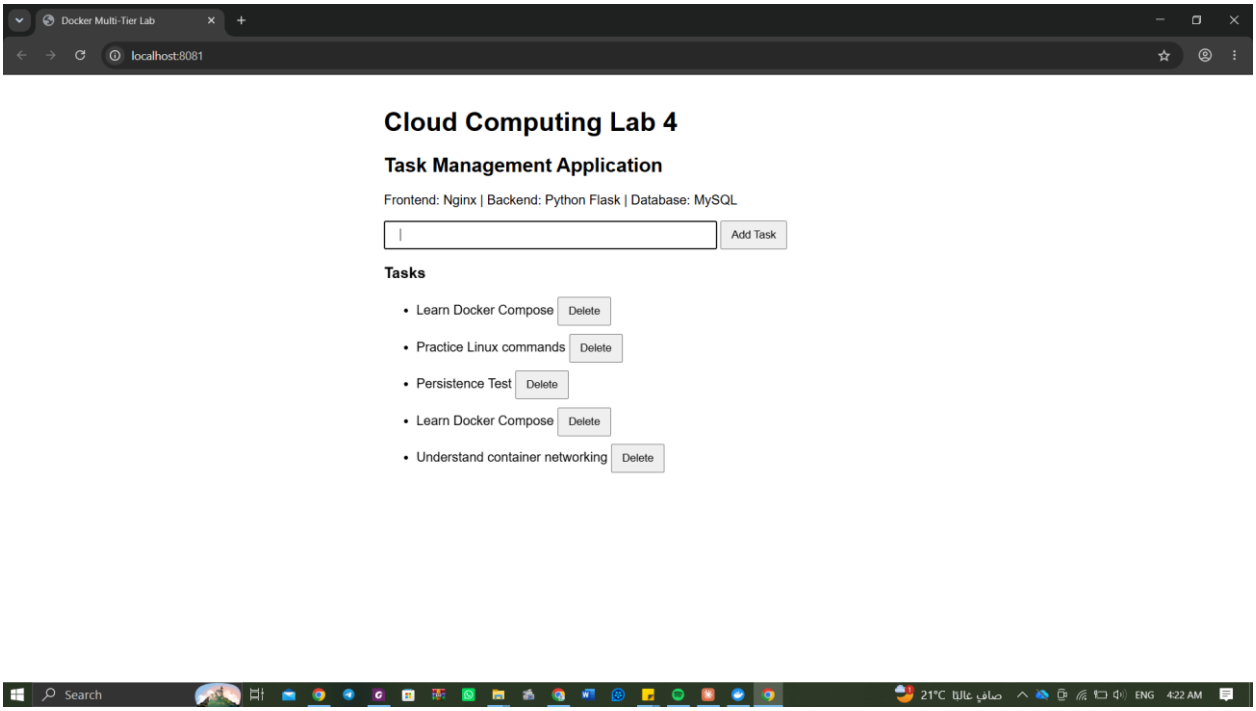


Figure 4: Screenshot 3 - the application at localhost:8081 after adding 'Learn Docker Compose' and 'Understand container networking' (a third task was deleted, a spaces-only task was refused)

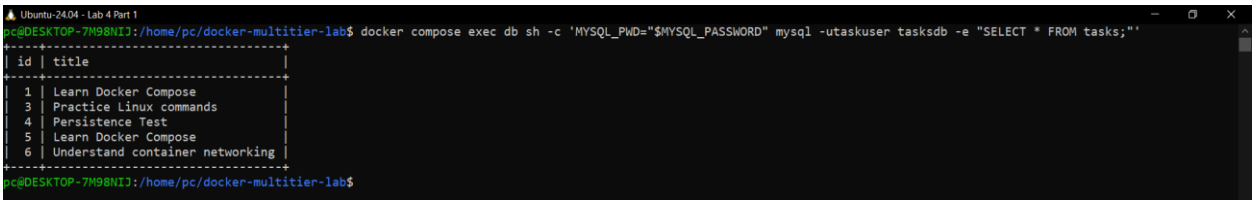


*Figure 5: Screenshot 4 - SELECT * FROM tasks in MySQL shows the same tasks as the browser*

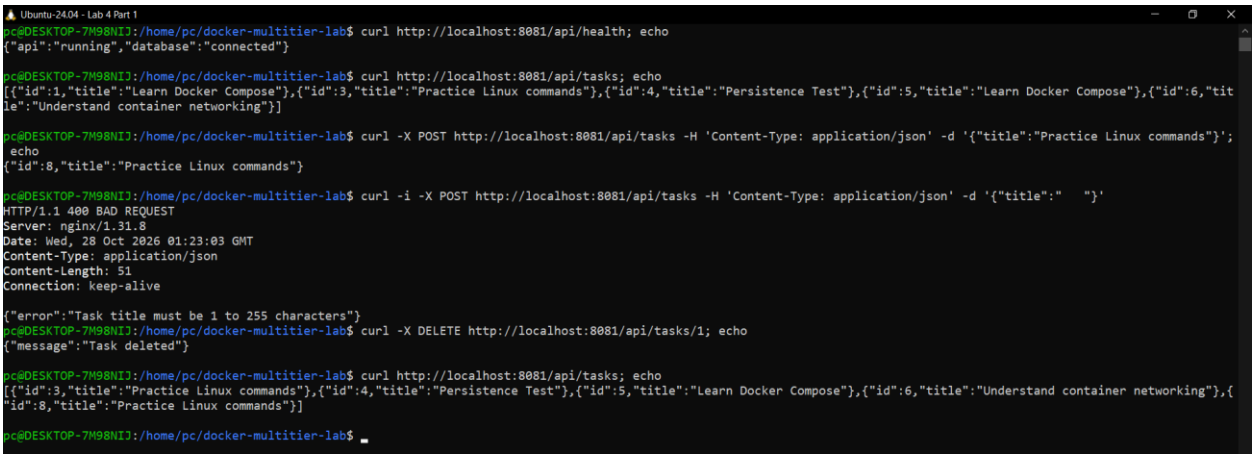


Figure 6: Extra - API tests with curl: health, list, add (201), spaces only (400), delete

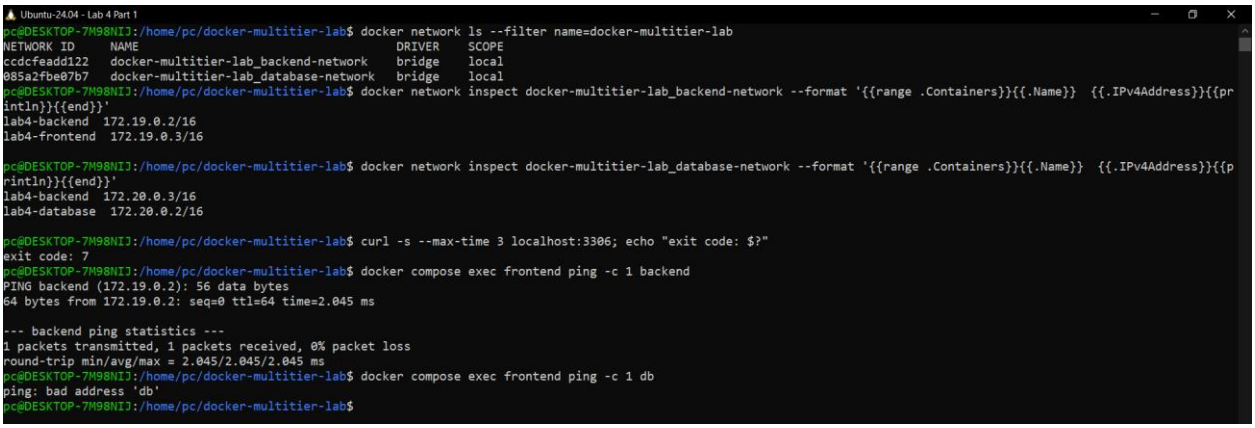


Figure 7: Screenshot 5 - networks and their containers; port 3306 is not reachable from the host; frontend reaches backend but not db

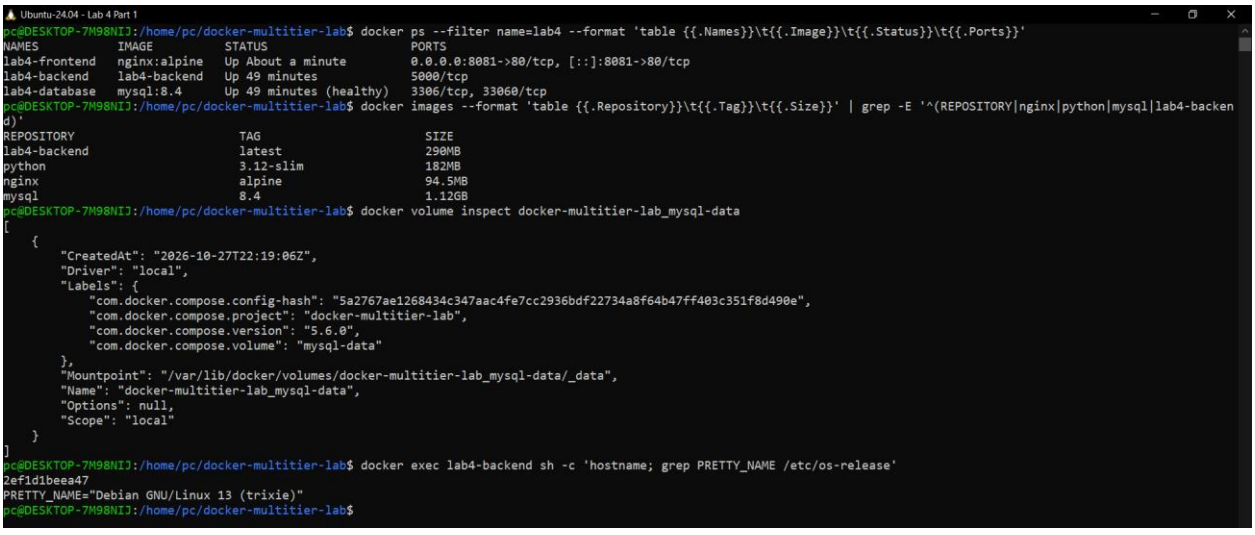


Figure 8: Extra - containers, images (lab4-backend is the one I built), the volume and a command inside the backend container

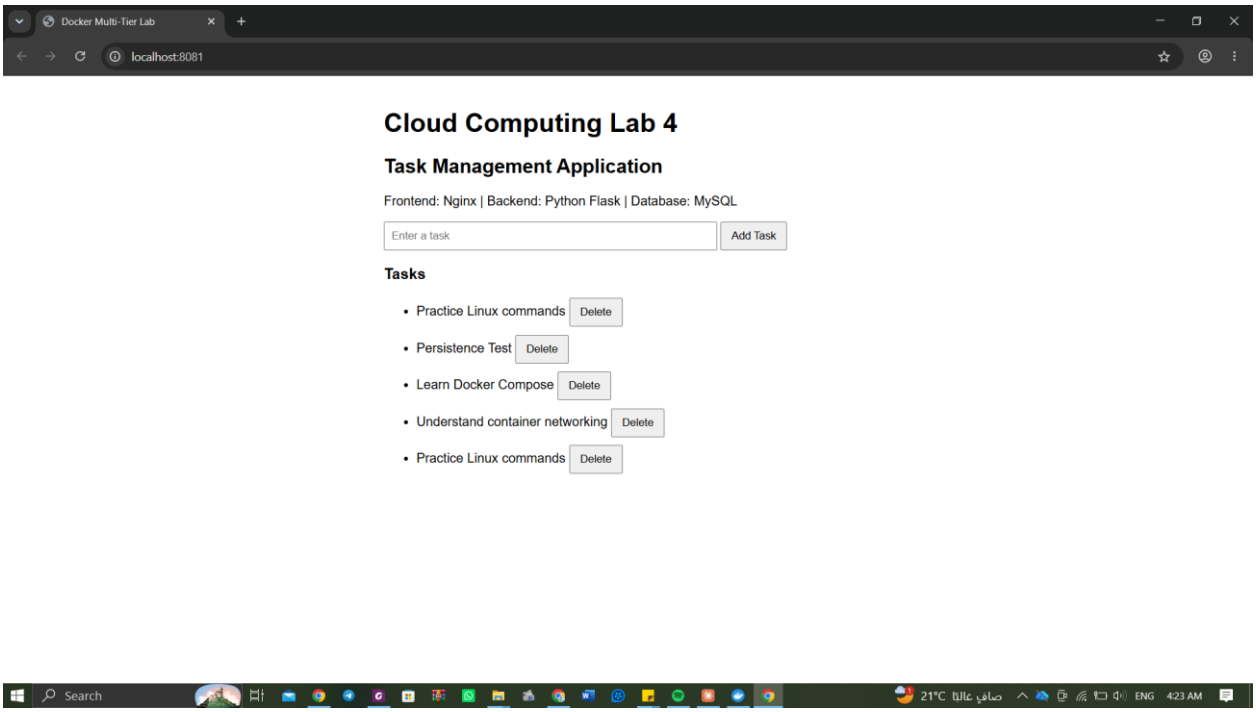


Figure 9: Before docker compose down: the task 'Persistence Test' is in the list

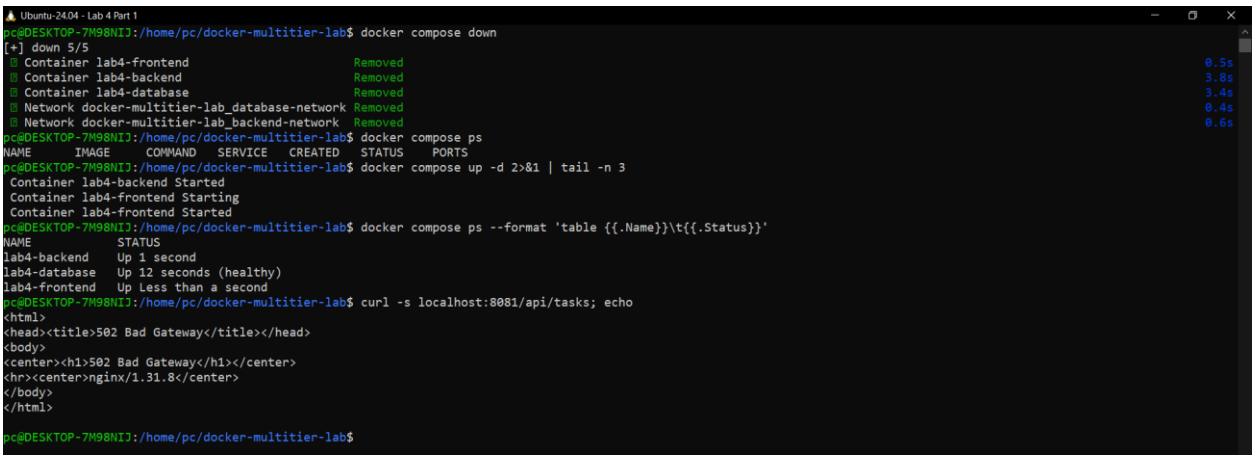


Figure 10: docker compose down, ps (nothing running), up -d, ps, and the API still returns the tasks

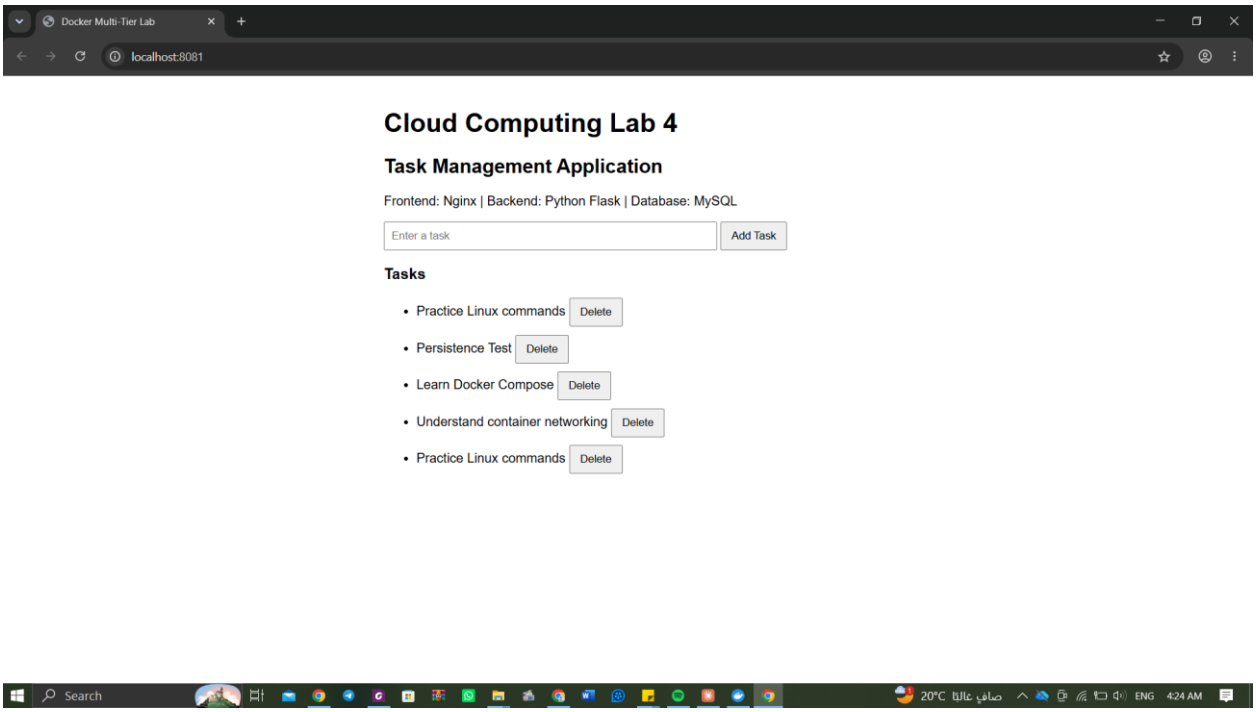


Figure 11: Screenshot 6 - the page after the restart still shows 'Persistence Test'

Experiments 1 to 3 - observations

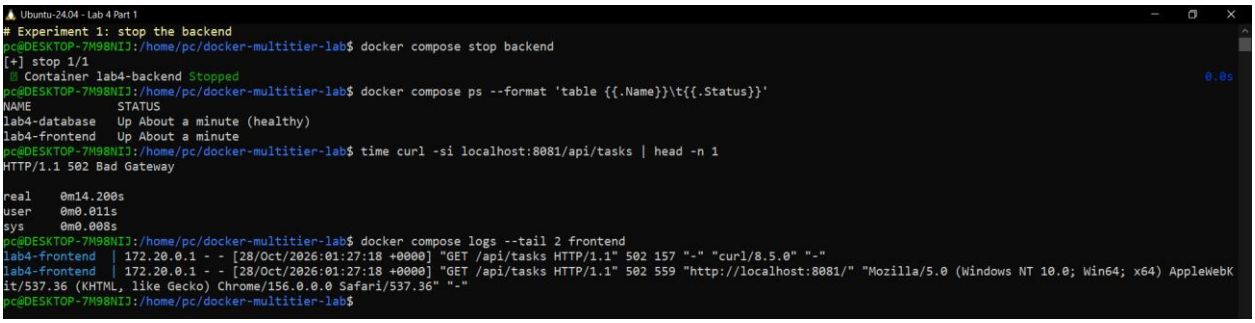


Figure 12: Experiment 1 - backend stopped: /api/tasks returns 502 Bad Gateway after about 14 s

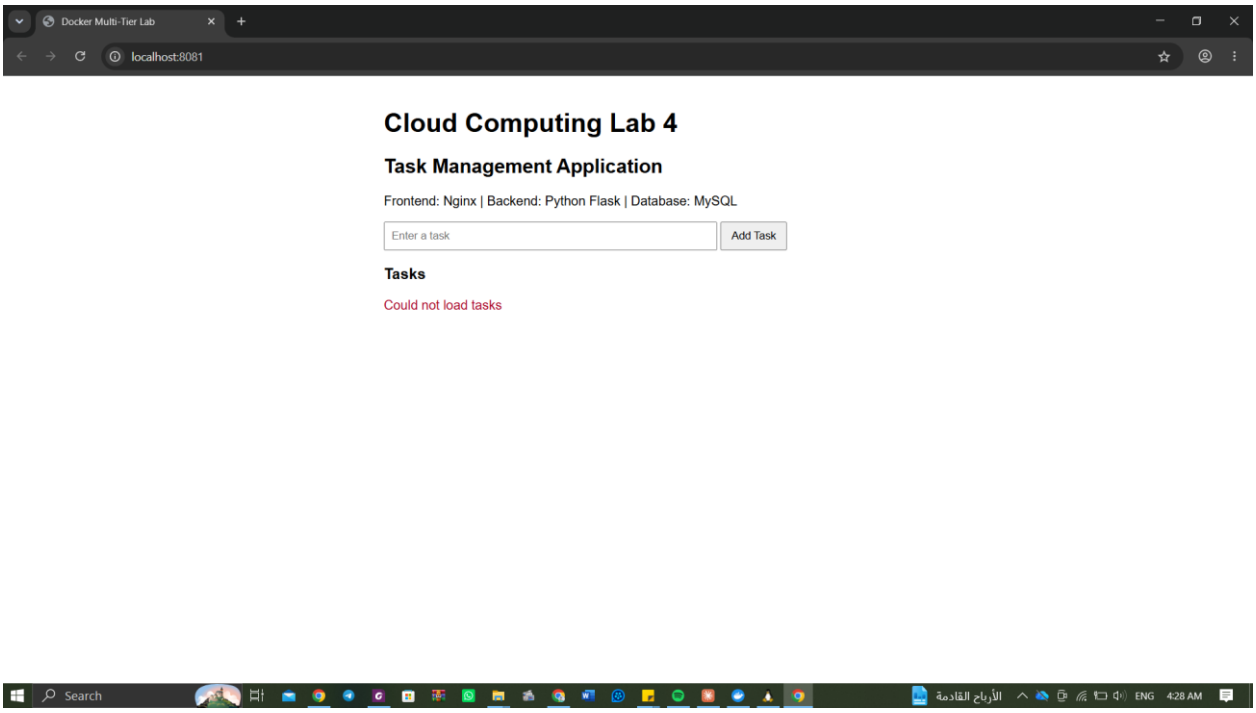


Figure 13: Experiment 1 - the page still loads but shows 'Could not load tasks'

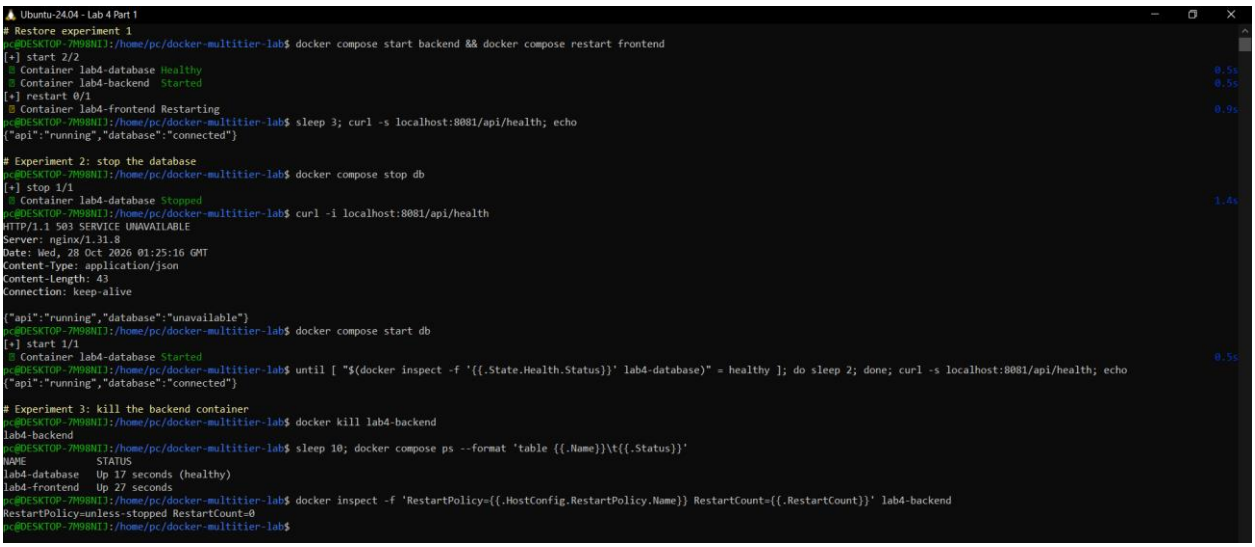


Figure 14: Restore of experiment 1, Experiment 2 (db stopped: HTTP 503) and Experiment 3 (docker kill)

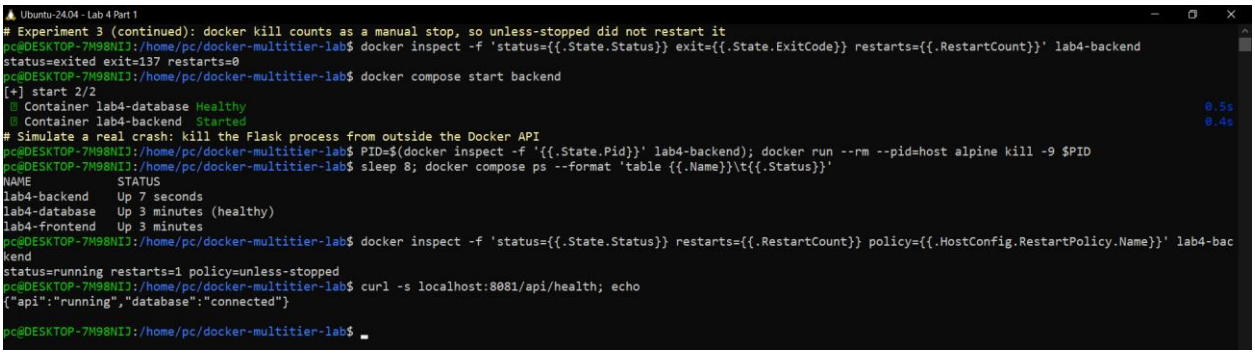


Figure 15: Experiment 3 continued - a real crash of the process is restarted automatically (RestartCount=1)

#	Action	Observation
1	docker compose stop backend	The page itself still loads, so the frontend (Nginx) is still running, but the task list is empty and the red message 'Could not load tasks' appears. curl shows 502 Bad Gateway: Nginx cannot reach its upstream. The failed tier is the backend. Restored with docker compose start backend and docker compose restart frontend (Nginx caches the backend's address), after which health is OK again.
2	docker compose stop db	curl -i localhost:8081/api/health returns HTTP 503 SERVICE UNAVAILABLE with {"api":"running","database":"unavailable"}. The API is alive but cannot connect to MySQL. After docker compose start db and waiting until it is healthy, health returns database: connected.
3	docker kill lab4-backend, wait 10 s, docker compose ps	On Docker Engine 29.8.2 the backend did NOT come back: docker kill is treated as a manual stop (exit code 137, RestartCount=0), and restart: unless-stopped never restarts a container that was stopped by the user. To check the policy itself I simulated a real crash by killing the Flask process from outside the Docker API (docker run --rm --pid=host alpine kill -9 <pid>): Docker restarted the container by itself within seconds (RestartCount=1). The setting responsible is restart: unless-stopped; it also brought all containers back when Docker Desktop was restarted.

Review questions

1. What is the difference between a Docker image and a container?

An image is a read-only, layered template that contains the file system and the start command (for example mysql:8.4). A container is a running instance of an image: it adds a thin writable layer, its own processes, network interfaces and name. Many containers can run from one image, and a container can be deleted and recreated from the image at any time.

2. Which images came from Docker Hub and which did you build? Why did the backend need a custom image?

nginx:alpine, python:3.12-slim, mysql:8.4 (and hello-world) came from Docker Hub. lab4-backend was built by me from backend/Dockerfile. The backend needed a custom image because no public image contains our app.py and its packages (Flask, mysql-connector-python); the Dockerfile starts from python:3.12-slim, installs requirements.txt, copies app.py and sets the start command. Nginx and MySQL only need configuration, which we mounted or passed as environment variables.

3. What does Nginx do besides serving the web page?

It is a reverse proxy: every request to /api/ is forwarded to http://backend:5000 and the answer is sent back to the browser. So the browser talks to one address only, the backend stays hidden inside Docker, and there are no cross-origin (CORS) problems.

4. Explain 8080:80. Why do the backend and database have no published ports?

The format is HOST_PORT:CONTAINER_PORT: connections to port 8080 on the laptop are forwarded to port 80 inside the Nginx container (on this PC 8081:80 because 8080 was taken). The backend and the database are only used by other containers, which reach them over the private Docker networks, so publishing them would only increase the attack surface: nobody outside should talk to MySQL or bypass the API.

5. How does the backend find the database?

compose.yaml sets DB_HOST=db for the backend, and app.py connects to that host name. 'db' is the Compose service name; Docker's internal DNS on database-network resolves it to the database container's IP.

6. Why can the frontend container not reach the database?

They share no network: the frontend is only on backend-network and the database only on database-network. The name 'db' does not even resolve from the frontend (ping: bad address 'db'), and there is no route between the two bridge networks. Only the backend is attached to both.

7. What is the difference between a Docker network and a Docker volume?

A network connects containers so they can communicate (virtual switch with DNS by service name); it is about traffic. A volume stores data outside the container's writable layer so it survives when the container is removed; it is about persistence.

8. Why did the tasks survive docker compose down? What would have happened with down -v?

down removes the containers and networks but keeps named volumes. MySQL's files are in the volume mysql-data (line '- mysql-data:/var/lib/mysql'), so the new database container found the old data. down -v also deletes the volume, so all tasks would be lost and MySQL would start with an empty database.

9. Why are the passwords in .env and not in app.py?

To keep secrets out of the source code and images: the same code can run with different passwords in different environments, .env can be kept out of Git, and changing a password does not need a code change or rebuild. app.py reads them from environment variables that Compose fills from .env.

10. Multi-container application vs multi-node cluster? What would change with one cloud VM per tier?

Here all containers run on one machine: if it fails everything fails, and the Docker networks exist only inside this host. A multi-node cluster spreads containers over several machines for availability and scaling and needs an orchestrator (Docker Swarm, Kubernetes) and overlay networking. With one VM per tier, the tiers would talk over the cloud's virtual network using private IPs or DNS names instead of Docker service names, so DB_HOST and the Nginx upstream would point to the other VMs; security groups / firewall rules would replace the Docker networks (web VM open on 80/443, backend reachable only from the web VM, database only from the backend VM); the database data would sit on that VM's disk or a managed database service, and each VM must be installed, patched, monitored and backed up separately.